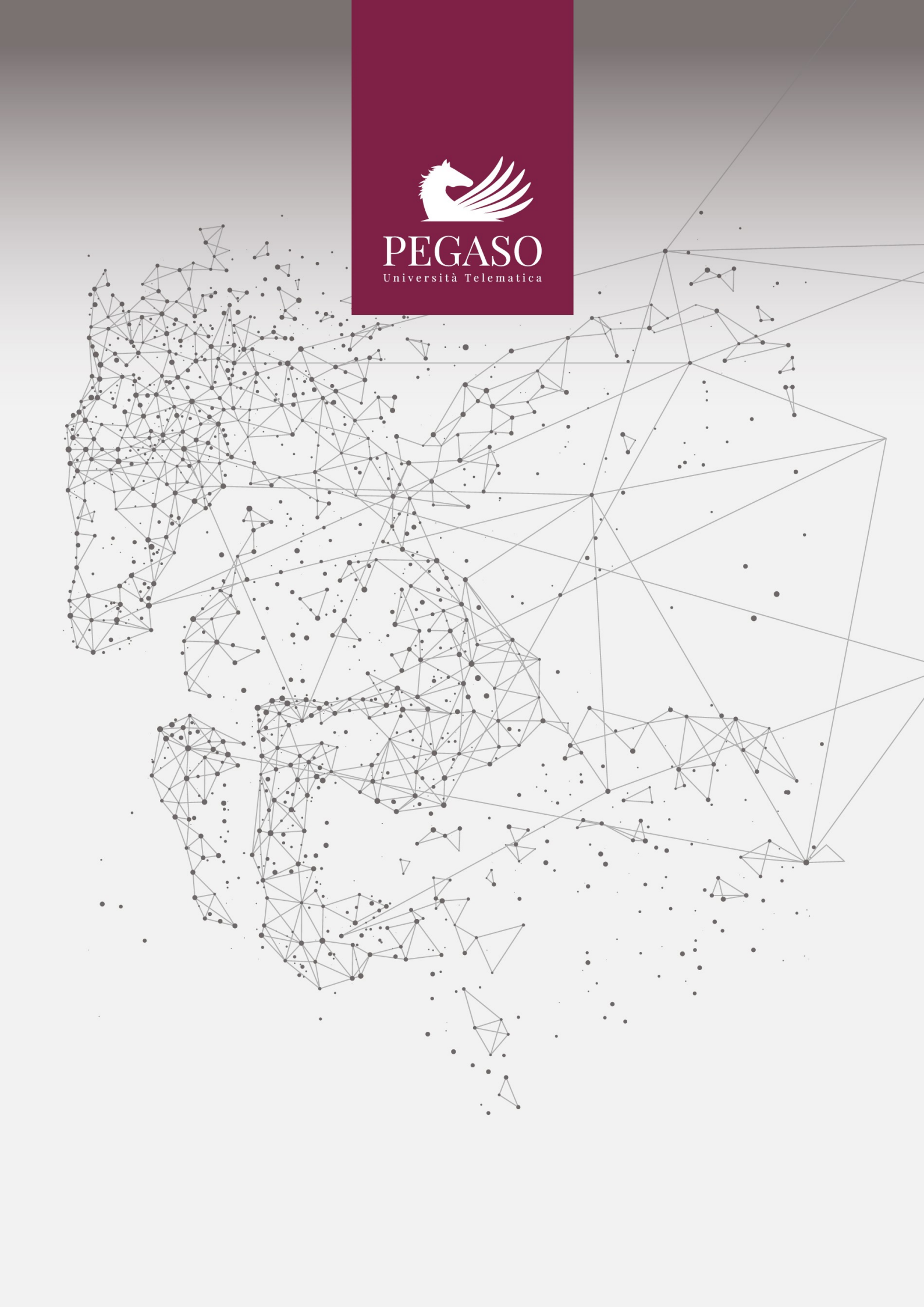

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. ISTRUZIONI ASSEMBLY MIPS BEQ E BNE 3

2. FORMATO DI TIPO I PER BEQ E BNE 7

3. INDIRIZZAMENTO RELATIVO AL PROGRAM COUNTER PER BEQ E BNE 11

BIBLIOGRAFIA 15

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Istruzioni Assembly MIPS beq e bne

Uno dei requisiti fondamentali della nozione di *algoritmo* è la possibilità di modificare l'ordine di esecuzione della lista di azioni mediante azioni contenute nella lista dette di *salto*:

ALGORITMO

una lista di AZIONI che devono essere eseguite da una MACCHINA:

1. ----- azione -----
2. ----- azione -----

…..

n. ----- azione -----

REQUISITI

- la lista di azioni è finita;
- ogni azione è comprensibile per la macchina senza ambiguità;
- ogni azione è eseguibile da parte della macchina;
- le azioni vengono svolte in sequenza a partire dalla prima;
- la lista può comprendere azioni, dette di salto, che cambiano l'ordine di esecuzione passando all'esecuzione di un'azione della lista diversa dalla successiva;
- la lista include una azione di arresto che ferma l'esecuzione da parte della macchina.

L'azione di salto può essere ***condizionata***, cioè essere effettuata solo se si verifica una specifica condizione, oppure ***incondizionata***, cioè essere eseguita sempre.

Per evitare ambiguità, le azioni di un Algoritmo che deve essere eseguito da un computer sono scritte seguendo le regole precise di un *Linguaggio di programmazione.*

Quindi, tutti i Linguaggi di programmazione prevedono istruzioni di salto la cui esecuzione cambia l'ordine di esecuzione sequenziale delle istruzioni del programma. Per effetto delle azioni di salto parti del programma possono essere ripetute, oppure possono essere tralasciate.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per poter eseguire una istruzione di salto la Sintassi del Linguaggio di programmazione deve dare la possibilità di identificare l'istruzione del programma alla quale saltare, cioè quella che diventa la prossima istruzione da eseguire.

I Linguaggi ad Alto Livello ed i Linguaggi Assembly permettono di assegnare ad una istruzione una ***ETICHETTA***, ovvero una sequenza di simboli che identifica univocamente l'istruzione nel programma, che è usata come **riferimento simbolico all'indirizzo di Memoria dove l'istruzione è memorizzata**.

Ricordiamo che la realizzazione Hardware di qualunque computer prevede un registro, denominato ***PROGRAM COUNTER*** (contatore di programma) che contiene l'indirizzo della prossima istruzione da eseguire.

Il contenuto di questo registro viene dinamicamente modificato automaticamente dall'Hardware del computer man mano che le istruzioni del programma vengono eseguite:

- se l'istruzione eseguita non è di salto, nel *Program Counter* viene posto l'indirizzo della parola di Memoria che contiene l'istruzione successiva,
- se, invece, l'istruzione eseguita determina un *salto*, nel *Program Counter* viene posto l'indirizzo della parola di Memoria che contiene l'istruzione alla quale saltare.

Le istruzioni di SALTO CONDIZIONATO del Linguaggio Assembly MIPS

Nel LINGUAGGIO ASSEMBLY MIPS sono disponibili due istruzioni di salto condizionato, con Codici Operativi beq e bne, in cui la condizione che deve essere verificata per saltare alla esecuzione di una istruzione diversa dalla successiva è rappresentata dalla relazione di uguaglianza o disuguaglianza dei valori contenuti in due registri.

Poiché tali istruzioni coinvolgono l'uso dei registri, riportiamo la tabella con gli indirizzi dei registri del processore utilizzati per i dati, indicando gli indirizzi numerici in Notazione decimale e la corrispondente notazione simbolica utilizzata nell'Assembly MIPS:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

simbolo	*indirizzo*	*simbolo*	*indirizzo*	*simbolo*	*indirizzo*
$t0	8	$s0	16	$t8	24
$t1	9	$s1	17	$t9	25
$t2	10	$s2	18		
$t3	11	$s3	19		
$t4	12	$s4	20		
$t5	13	$s5	21		
$t6	14	$s6	22		
$t7	15	$s7	23		

Le istruzioni del Linguaggio Assembly MIPS con Codici Operativi *beq* ed *bne* hanno due operandi rappresentati da sequenze di 32 bit contenute in due registri del processore. Eseguono un salto ad una istruzione specificata mediante una *etichetta* se i valori contenuti nei due registri sono uguali (*beq*) oppure se sono diversi (*bne*).

La seguente istruzione in **Assembly MIPS** fornisce un esempio della sintassi dell'istruzione *beq*:

beq $t0, $t5, *nome_etichetta*

Vincoli sintattici:

- *Codice Operativo* ***beq*** (***b****ranch if* ***eq****ual*, dirama se uguale) che specifica l'operazione;
- dopo il *Codice Operativo* vengono indicati gli **indirizzi dei due registri di cui confrontare il contenuto**, nell'esempio $t0, $t5;
- separato da una virgola segue una sequenza alfanumerica *nome_etichetta* che rappresenta **l'ETICHETTA** che precede l'istruzione a cui **saltare se il contenuto dei due registri è uguale**, identificandone univocamente la posizione nella lista delle istruzioni del programma scritto in Linguaggio Assembly.

L'istruzione *bne* (***b****ranch if* ***n****ot* ***e****qual*, dirama se non uguale) ha una sintassi del tutto simile, la differenza è in fase di esecuzione perché il salto avviene se il contenuto dei due registri è diverso.

L'esecuzione di una istruzione di salto condizionato non cambia il contenuto della Memoria e dei registri del processore.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Ciò che può cambiare è il contenuto del registro denominato *Program Counter* (contatore di programma) che contiene l'indirizzo della prossima istruzione da eseguire. Se la condizione di salto è verificata il contenuto di questo registro viene modificato scrivendo l'indirizzo di Memoria della istruzione cui si deve saltare.

Le seguenti istruzioni in Assembly MIPS presentano un esempio di utilizzazione dell'istruzione di salto condizionato *beq*:

```
beq  $t0, $t5,  no_incremento
addi  $s3, $s3, 1
no_incremento        sw   $s3,  5($s7)
```

l'etichetta *no_incremento* precede il *Codice Operativo* dell'istruzione a cui saltare.

Se il contenuto del registro di indirizzo $t0 è uguale al contenuto del registro di indirizzo $t5, si salta all'esecuzione dell'istruzione *sw* con etichetta *no_incremento* e quindi non viene eseguita l'istruzione *addi*.

Invece, se la condizione non è verificata perché i contenuti dei registri sono diversi, si continua in sequenza eseguendo l'istruzione *addi* prima dell'istruzione *sw*.

Come effetto delle due scelte di esecuzione alternativa si ha che in un caso l'istruzione *sw* scrive in Memoria il valore contenuto in $s3 incrementato di 1, nell'altro lo scrive senza incrementarlo.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Formato di Tipo I per beq e bne

Come già osservato nel paragrafo precedente, per poter eseguire una istruzione di salto la Sintassi del Linguaggio di programmazione deve dare la possibilità di identificare l'istruzione del programma alla quale saltare, cioè quella che diventa la prossima istruzione da eseguire.

Nei Linguaggi ad Alto Livello e nei Linguaggi Assembly questa richiesta viene soddisfatta dando la possibilità di assegnare ad una istruzione una ***ETICHETTA***, ovvero una sequenza di simboli che identifica univocamente l'istruzione, che quindi può essere usata come riferimento alla sua posizione nel programma.

Nel LINGUAGGIO MACCHINA **questa possibilità non c'è**, perché per individuare l'istruzione alla quale saltare bisogna fornire **l'indirizzo** della **parola di Memoria** che contiene l'istruzione. Tale indirizzo viene calcolato mediante le regole della Modalità di Indirizzamento associata all'istruzione di salto utilizzando le informazioni fornite nei campi del Formato dell'istruzione.

Prima di introdurre le istruzioni in Linguaggio Macchina MIPS che traducono le istruzioni Assembly di salto condizionato *beq* e *bne*, ricordiamo sinteticamente la struttura di base di tale Linguaggio.

La sequenza binaria che rappresenta una istruzione in Linguaggio Macchina permette di ricavare tutte le informazioni che definiscono sia l'azione associata all'istruzione da eseguire, sia le modalità per il reperimento dei dati.

Queste informazioni sono codificate mediante due fondamentali nozioni che sono alla base della definizione di tutti i Linguaggi Macchina e che caratterizzano ogni istruzione:

- il Formato dell'istruzione,
- la Modalità di Indirizzamento dell'istruzione.

La SEQUENZA BINARIA che rappresenta l'istruzione in **Linguaggio Macchina** è suddivisa in sottosequenze, dette CAMPI, di lunghezza e posizione fissata i cui contenuti forniscono specifiche informazioni determinate univocamente dalle regole sintattiche dell'istruzione. Questa strutturazione, detta **FORMATO**, è solo una organizzazione concettuale e non determina nessuna suddivisione a livello fisico della sequenza. La lunghezza dell'istruzione del Linguaggio Macchina MIPS è di 32 Bit.

Le **ISTRUZIONI DI SALTO CONDIZIONATO che traducono in LINGUAGGIO MACCHINA MIPS le istruzioni beq e bne** utilizzano il **Formato di Tipo I** per la codifica delle relative informazioni.

Il Formato di Tipo I del Linguaggio Macchina MIPS deve la sua denominazione al fatto che un campo della sequenza binaria che rappresenta una istruzione con questo Formato contiene un numero intero in Notazione in complemento a 2.

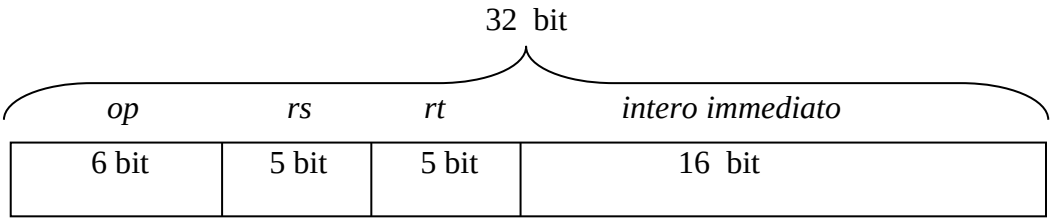
PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Questo numero fornisce il valore di un dato dell'istruzione detto **operando immediato**. Quindi la lettera I è l'iniziale della parola *Immediato*.

Il seguente schema indica la **partizione** concettuale (non fisica) in sottosequenze della sequenza di 32 bit dell'istruzione.

Formato di Tipo I



op: 6 bit, campo del *Codice Operativo* presente in tutte le istruzioni. Il valore contenuto indica l'operazione da eseguire ed il Formato dell'istruzione;

rs: 5 bit, indirizzo di registro;

rt: 5 bit, indirizzo di registro;

intero immediato: 16 bit, intero in Notazione in complemento a 2.

Illustriamo le regole di traduzione in Linguaggio Macchina dell'istruzione *beq* utilizzando l'esempio in Assembly MIPS introdotto nel paragrafo precedente:

beq $t0, $t5, *no_incremento*

Il *Codice Operativo* è codificato nel campo di 6 bit comune a tutti i Formati. Per l'istruzione ***beq*** il campo ***Codice Operativo*** contiene il **valore decimale 4**, rappresentato in Notazione binaria dalla sequenza 000100, e fornisce tutte le informazioni che specificano l'operazione da eseguire ed il Formato dell'istruzione.

Gli indirizzi dei due registri che contengono i valori da confrontare per stabilire se la condizione di salto è verificata sono dati da sequenze di 5 bit poste nei campi *rs* ed *rt* del Formato di Tipo I.

Nel campo *immediato* di 16 bit viene posto un numero intero con segno in Notazione in complemento a 2 che rappresenta il numero di istruzioni da saltare, in avanti (numero positivo) oppure tornando indietro (numero negativo), rispetto alla posizione dell'istruzione successiva a *beq*, che è quella da eseguire se il salto non avviene.

In base alla tabella delle corrispondenze tra notazione simbolica e valore dell'indirizzo si ha che gli indirizzi dei due registri da confrontare $t0, $t5 sono dati da:

$t0 ⟷ 8 ⟷ 01000

$t5 ⟷ 13 ⟷ 01101

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Le due sequenze binarie che forniscono il valore degli indirizzi occupano i campi di 5 bit del Formato di Tipo I dell'istruzione conservando lo stesso ordine che hanno nell'istruzione Assembly.

Quindi l'esempio di istruzione *beq* dell'Assembly MIPS viene tradotto nella seguente istruzione in Linguaggio Macchina MIPS con Formato di Tipo I:

beq $t0, $t5, *no_incremento*

op	*rs*	*rt*	*intero immediato*
4	8	13	numero delle istruzioni da saltare

la cui rappresentazione binaria è:

000100	01000	01101	numero delle istruzioni da saltare

Il contenuto del campo *immediato* è il **numero delle istruzioni da saltare.** Questo valore viene determinato in fase di traduzione dal programma Assembler, che lo calcola in base alla posizione della ETICHETTA *nome_etichetta* nella lista delle istruzioni del programma.

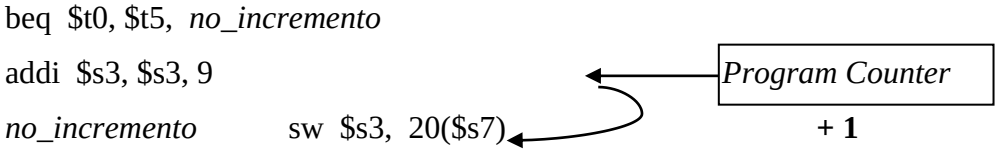
Illustriamo il calcolo di questo valore utilizzando l'esempio dato nel paragrafo precedente:

beq $t0, $t5, *no_incremento*
addi $s3, $s3, 9
no_incremento sw $s3, 20($s7)

Quando *beq* è in esecuzione il ***Program Counter*** contiene l'indirizzo della istruzione successiva *addi*.

Il programma **Assembler** nella fase di **traduzione in Linguaggio Macchina**, calcola il numero di istruzioni da aggiungere (positivo se in più, negativo se in meno) per arrivare all'ETICHETTA *no_incremento* dell'istruzione *sw* rispetto all'istruzione successiva, indicata nel *Program Counter*, e lo pone nel campo *immediato.*

Nell'esempio bisogna aggiungere il valore 1 al *Program Counter* poiché l'ETICHETTA *no_incremento* figura davanti all'istruzione *sw*, ad una distanza di 1 in avanti rispetto all'istruzione successiva *addi*, il cui indirizzo di Memoria è contenuto nel *Program Counter*, come mostrato nel seguente schema:



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Quindi il programma traduttore Assembler, calcola il valore 1 di questa distanza e lo pone nel campo *immediato* dell'istruzione in Linguaggio Macchina che traduce *beq*:

op	*rs*	*rt*	*intero immediato*
4	8	13	1

la cui rappresentazione binaria è:

000100	01000	01101	0000000000000001

e quindi la sequenza dei **32 bit dell'istruzione in Linguaggio Macchina che traduce *beq*** è:

00010001000011010000000000000001

Le regole sintattiche per la traduzione in Linguaggio Macchina dell'istruzione Assembly ***bne*** di salto condizionato su disuguaglianza, che ha ***Codice Operativo* 5**, in Notazione binaria 000101, sono analoghe a quelle descritte per *beq*.

Il Linguaggio Assembly può prevedere ulteriori istruzioni di salto condizionato basate sulla verifica di condizioni diverse, ad esempio sul confronto dei valori contenuti nei registri per verificare la relazione *strettamente minore* effettuando il salto se risulta verificata.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Indirizzamento relativo al Program Counter per beq e bne

Nel registro *Program Counter* (Contatore di Programma) è contenuto l'indirizzo della prossima istruzione da eseguire, che di norma è la successiva a quella in esecuzione.

Le istruzioni di salto condizionato, quando la condizione è verificata, eseguono il salto ponendo nel *Program Counter* l'indirizzo della parola di Memoria che contiene l'istruzione alla quale saltare.

Questo indirizzo, detto di **indirizzo di salto**, è ottenuto ADDIZIONANDO al valore contenuto nel *Program Conter* il numero di locazioni di Memoria corrispondente al numero di istruzioni da saltare contenuto nel campo *immediato* del Formato di Tipo I dell'istruzione in Linguaggio Macchina.

L'addizione è eseguita dall'Unità Aritmetico-Logica (ALU), che è strutturata per ricevere operandi rappresentati da sequenze di 32 bit. Quindi, è necessario effettuare una ESTENSIONE DEL SEGNO per aumentare a 32 bit la lunghezza della sequenza nel campo *immediato*. L'estensione del segno viene fatta aggiungendo a sinistra della sequenza i 16 bit mancanti, con valore uguale a quello del bit più significativo (bit di segno). Grazie alle proprietà della Notazione in Complemento a 2, i bit aggiunti alla sequenza non cambiano il valore rappresentato.

L'ampiezza massima del salto possibile deriva dall'intervallo di rappresentazione degli interi in Notazione in complemento a 2, che con sequenze di 16 bit è compreso tra il minimo negativo $-2^{16-1}=-2^{15}$, ed il massimo positivo $2^{16-1}-1=2^{15}-1$.

Inoltre, bisogna ricordare che una istruzione memorizzata in Memoria occupa 4 Byte, in quando la Memoria è un blocco di registri dove ogni locazione contiene un Byte (8 bit) e ogni istruzione del Linguaggio Macchina MIPS è rappresentata da 32 bit.

Quindi, il valore rappresentato dal campo *immediato* esteso di segno, che rappresenta il numero di istruzioni da saltare, viene MOLTIPLICATO PER 4 (numero di Byte occupato da ogni istruzione) prima di essere sommato al *Program Counter*. Questa operazione viene effettuata automaticamente dall'Hardware del computer.

Queste osservazioni sono riassunte nella regola di calcolo dell'indirizzo di salto condizionato per le istruzioni *beq* e *bne* data dalla **Modalità di Indirizzamento relativo al *Program Counter***, che utilizza il numero intero contenuto nei 16 bit del campo *immediato* del Formato di Tipo I dell'istruzione, e l'indirizzo rappresentato dal valore contenuto nei 32 bit del *Program Counter*:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

INDIRIZZAMENTO relativo al *Program Counter*

Indirizzo di Memoria dell'istruzione a cui saltare =

Program Counter + 4 x *campo immediato* esteso di segno a 32 bit.

Illustriamo la regola che è analoga per *beq* e *bne* considerando l'esempio relativo a *beq* precedentemente descritto:

```
beq  $t0, $t1,  no_incremento
addi  $s3, $s3, 9
no_incremento        sw  $s3,  5($s7)
```

dove la traduzione in Linguaggio Macchina dell'istruzione *beq* è:

000100	01000	01101	0000000000000001

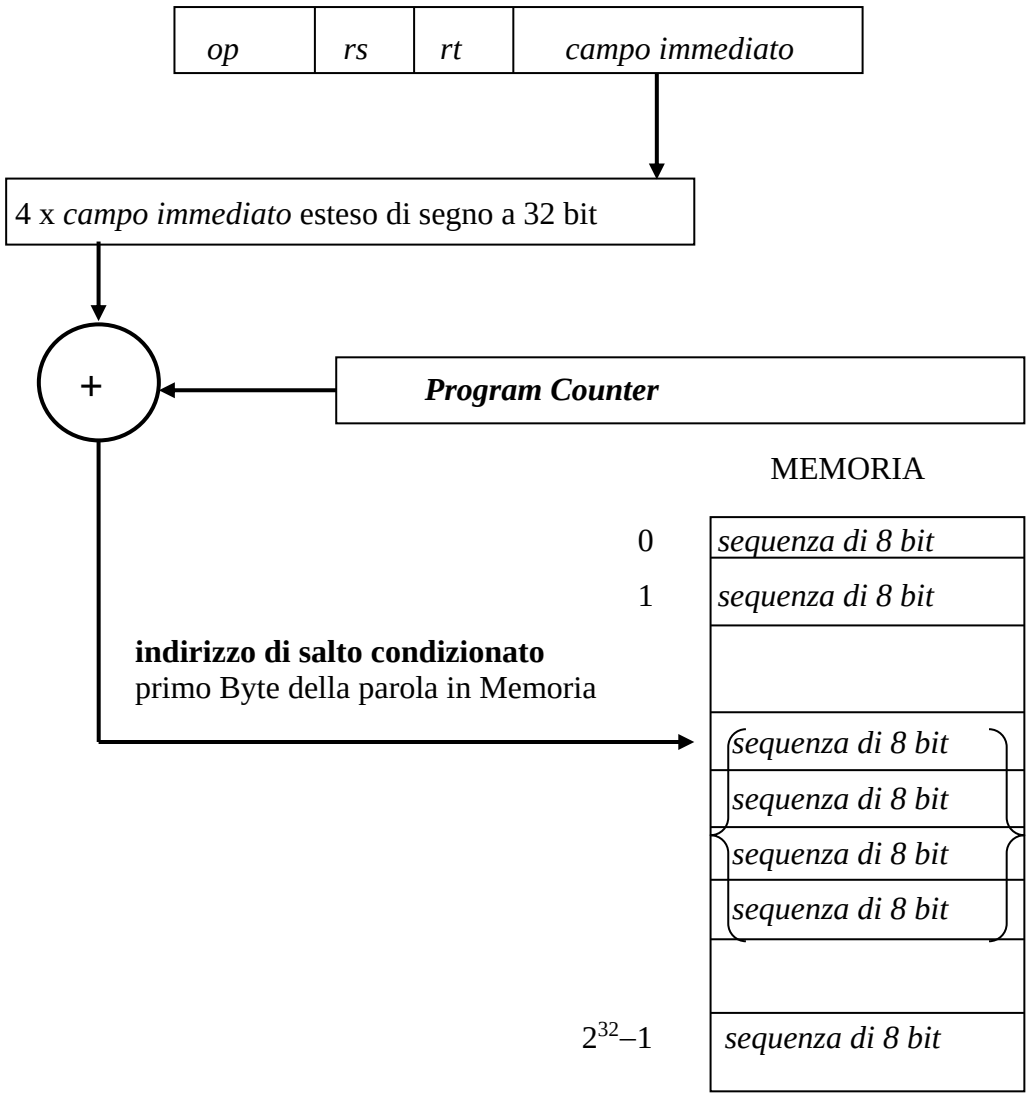
Se la condizione di salto è verificata, al contenuto del *Program Counter* viene addizionato il valore 4x1, esteso di segno a 32 bit, poiché il campo *immediato* contiene 1 come numero di istruzioni da saltare.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La seguente figura fornisce una rappresentazione schematica della Modalità di *Indirizzamento relativo al Program Counter*:

MODALITÀ di Indirizzamento relativo al *Program Counter*



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Le seguenti Tabelle riassumono regole sintattiche e Modalità di indirizzamento di istruzioni in Linguaggio Macchina MIPS:

Linguaggio Macchina MIPS

	6 bit	5 bit	5 bit	5 bit	5 bit	6 bit	esempio
istruzione	*Tipo*	*op*	*rs*	*rt*	*rd*	*shamt*	*funct*	
add	R	0	$s1	$s2	$t0	0	32	add $t0, $s1, $s2
sub	R	0	$s1	$s2	$t0	0	34	sub $t0, $s1, $s2
and	R	0	$s1	$s2	$t0	0	36	and $t0, $s1, $s2
or	R	0	$s1	$s2	$t0	0	37	or $t0, $s1, $s2
sll	R	0	0	$s2	$t0	7	0	sll $t0, $s2, 7
srl	R	0	0	$s2	$t0	7	2	srl $t0, $s2, 7
addi	I	8	$s3	$t0	–9			addi $t0, $s3,–9
lw	I	35	$s3	$t0	–9			lw $t0, –9($s3)
sw	I	43	$s3	$t0	–9			sw $t0, –9($s3)
beq	I	4	$t0	$s3	istruzioni da saltare			beq $t0, $s3, *nome*
bne	I	5	$t0	$s3	istruzioni da saltare			bne $t0, $s3, *nome*

le relative Modalità di indirizzamento sono:

istruzione	*Formato Tipo*	*Modalità di Indirizzamento*
add	R	tramite registro
sub	R	tramite registro
and	R	tramite registro
or	R	tramite registro
sll	R	tramite registro
srl	R	tramite registro
addi	I	immediato
lw	I	tramite Base e Offset
sw	I	tramite Base e Offset
beq	I	relativo al *Program Counter*
bne	I	relativo al *Program Counter*

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- D. A. Patterson, J. L. Hennessy, "Struttura e progetto dei calcolatori", Zanichelli.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).